DOST-FNRI ELIBRARY

Deployment checklist

Local development · Staging · Production

How to install and update the DOST-FNRI eLibrary on a developer's machine (Part A), on the FNRI MIS staging server (Part B) and on the public production server (Part C). Part D holds what all three share: settings, web server, PHP, scheduler, backups, upgrades, Google sign-in and troubleshooting.

> **Checked 27 October 2026** (commit `dbc426d`): installed from a clean clone onto an empty database, then switched to production mode without development packages. Every command in Parts A to C ran, the sample accounts signed in, and the admin pages, search, sitemap and maintenance mode worked.

Part	Where	You end up with
A. Local	a developer's PC: Windows with Laragon, or Linux	the app at `http://localhost:8000` or `http://elibrary.test`, sample accounts, the tests
B. Staging	the FNRI MIS test server	a private copy on HTTPS for testing, inside the FNRI network
C. Production	the public server	the live OPAC on its own domain
D. Reference	all of them	settings, web server, PHP, scheduler, backups, upgrades, Google sign-in, troubleshooting

Made on 28 October 2026 from `docs/implemented/DEPLOYMENT_CHECKLIST.md` in the smart-elibrary repository. That Markdown file is the source: edit it, then make this PDF again.

Contents

- What makes up the eLibrary

Part A — Local development

- A1. Install the tools (once per machine)
- A2. Get the code
- A3. Configure `.env`
- A4. Create the database
- A5. Files and search
- A6. Run it and sign in
- A7. Tests and the security checks
- A8. After pulling new code

Part B — Staging

- B1. What the server needs
- B2. First deploy
- B3. Keep staging private
- B4. Accounts
- B5. Check it
- B6. Update staging
- B7. Start staging over

Part C — Production

- C1. How production differs from staging
- C2. Before go-live
- C3. First deploy and go-live
- C4. Updating production
- C5. Rolling back

Part D — Reference

- D1. Environment variables
- D2. Web server
- D3. PHP settings
- D4. Scheduler
- D5. Moving an existing database forward
- D6. Smoke test
- D7. Backups and restore
- D8. Google Search Console
- D9. The AI half (optional)
- D10. Google sign-in and the "I'm human" check
- D11. Troubleshooting
- D12. Known gaps

What makes up the eLibrary

Piece	Needed	Notes
The Laravel app (this repository)	always	PHP **8.5.12 or newer** (Composer refuses older), Laravel 13
MySQL **8.0 or newer**	always	one database, `elibrarydb`, `utf8mb4` / `utf8mb4_unicode_ci`
Meilisearch **1.12 or newer**	always	the OPAC search; `docs/implemented/MEILISEARCH_SETUP.md`
The scheduler	staging, production	rebuilds the sitemap (03:30) and checks the search index (03:45) every night
The AI half (`hermes-with-rag`)	optional	only for AI mode, which is off until switched on in Admin → Settings (D9)
A Google Cloud project and Cloudflare Turnstile	for "Continue with Google" and the "I'm human" check on sign-up	each off while its keys are empty (D10)

- **PHP extensions:** ctype, curl, dom, fileinfo, gd, iconv, mbstring, openssl, pdo_mysql, simplexml, tokenizer, xml.
- **Composer 2.**
- **Node.js 20.19+ or 22.12+:** only to rebuild the CSS and JavaScript. The built files (`public/build`) are in the repository, so a server never needs Node.
- **No queue worker:** `QUEUE_CONNECTION=sync`, so the search index updates as records are saved.

Part A — Local development

A1. Install the tools (once per machine)

- **Windows:** Laragon with Apache and MySQL 8, PHP 8.5.12 (Menu → PHP; then delete the older PHP folders under `C:\laragon\bin\php`), Git, Composer, and Docker Desktop for Meilisearch (or Meilisearch's own `.exe`).
- **Linux:** PHP 8.5 (CLI) with the extensions above, MySQL 8, Composer, Git, and Meilisearch, standalone or in Docker (MEILISEARCH_SETUP.md).

A2. Get the code

```
git clone git@github.com:UrekMazino/smart-elibrary.git elibrary   # Laragon: inside
C:\laragon\www
cd elibrary
composer install
cp .env.example .env                        # Windows: copy .env.example .env
php artisan key:generate
git config core.hooksPath .githooks         # secret scan before each commit, checks before each
push
```

A3. Configure `.env`

The template works locally as it is, except for these keys (all keys: D1):

Key	Set to
`APP_URL`	`http://localhost:8000` (`php artisan serve`) or `http://elibrary.test` (Laragon)
`DB_USERNAME`, `DB_PASSWORD`	your MySQL account (Laragon: `root`, empty)
`SEED_USER_PASSWORD`	a password for the sample accounts (remove it after A4)
`SESSION_SECURE_COOKIE`	`false` **only** on Laragon's `http://elibrary.test`, where sign-in fails otherwise; keep `true` on `http://localhost`
`MEILI_MASTER_KEY`, `MEILISEARCH_KEY`	the same random value, 16 characters or more
`PASSWORD_BREACH_CHECK`	`false` if the PC has no internet access
`TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	as in the template: Cloudflare's test keys, which pass every "I'm human" check; empty turns the check off
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	the local-and-staging OAuth client (D10); empty hides "Continue with Google"

Mail stays in `storage/logs/laravel.log` (`MAIL_MAILER=log`): password-reset links are read from there.

"Continue with Google" works locally only on `http://localhost:8000` (`php artisan serve`, with `APP_URL` the same): Google accepts plain `http://` return addresses for `localhost` alone, so not on Laragon's `http://elibrary.test`.

A4. Create the database

```
mysql -u root -p -e "CREATE DATABASE elibrarydb CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci"
php artisan migrate --seed
```

This builds every table and loads the reference data (MARC tags, materials, roles and permissions) and three sample accounts, all with `SEED_USER_PASSWORD` (or a random password printed once):

Account	Email	Role
Developer Account	`developer@example.org`	developer: everything
Super Administrator	`superadmin@example.org`	super admin
Sample Patron	`patron@example.org`	patron

To work on a copy of real data instead: import a dump into the empty database (`mysql -u root -p elibrarydb < dump.sql`), then `php artisan migrate` and the one-time commands in D5.

A5. Files and search

```
php artisan storage:link                    # thumb, fullpage, news covers -> storage/app/public
docker compose up -d                        # Meilisearch 1.12 on 127.0.0.1:7700 (or
MEILISEARCH_SETUP.md)
php artisan scout:sync-index-settings       # the filters and sorts the search page uses
php artisan scout:import "App\Models\ViewCatalog"
```

On Windows, run `storage:link` from a terminal opened as administrator if it can't create the links.

A6. Run it and sign in

- `php artisan serve` → http://localhost:8000, or Laragon's Apache → http://elibrary.test (Laragon's automatic host uses the `public` folder).
- Sign in at `/login` as `superadmin@example.org`.
- The scheduler is optional locally: `php artisan schedule:work` in a terminal, or once on Windows `powershell -ExecutionPolicy Bypass -File scripts\register-scheduler-task.ps1`.

A7. Tests and the security checks

```
mysql -u root -p -e "CREATE DATABASE elibrarydb_test CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci"   # once
php artisan test                            # or vendor/bin/phpunit; refuses any database not
named *_test
composer security                           # what the pre-push hook runs: gitleaks, audits,
Semgrep, tests
```

`composer security` needs gitleaks and Semgrep (`pip install semgrep==1.178.0`). The pre-push hook stops a push when a check fails, and needs the development packages: after a `composer install --no-dev`, run `composer install` again before pushing.

A8. After pulling new code

```
git pull
composer install
php artisan migrate
php artisan optimize:clear
```

Then any one-time command the commit names (D5). After changing anything under `resources/css` or `resources/js`: `npm ci && npm run build` (Node 20.19+ or 22.12+), and commit `public/build`.

Part B — Staging

A private copy for testing before production: the same software and settings as production, but inside the FNRI network, hidden from search engines, with test mail and test data.

Where its code comes from: the FNRI GitLab, project `root/digitalfirst` (https://gitlab.fnri.dost.gov.ph/root/digitalfirst.git), branch **`staging`**. GitHub (`UrekMazino/smart-elibrary`) stays the main repository: work is merged into `main` there, and a release to staging merges `main` into `staging` and pushes it to GitLab (B6).

B1. What the server needs

What the MIS Unit provides (the "Programmer Requirements for Staging Environment" form asks for this):

- **Linux** (Ubuntu 24.04 LTS preferred) or Windows Server.
- **PHP 8.5.12 or newer**, FPM and CLI, with the extensions in the list above; **Composer 2**.
- **Apache 2.4** (`rewrite`, `headers`, `ssl`, `proxy_fcgi`) or **nginx**: D2.
- **MySQL 8.0 or newer**, and **Meilisearch 1.12 or newer** running as a service.
- **A TLS certificate** for the staging host name: sign-in only works over HTTPS.
- **Ports:** 443 and 80 (redirects to 443) for users; 22 for the developer only; 3306 (MySQL), 7700 (Meilisearch) and 8090 (AI gateway, if used) reachable from the server itself only.
- **Access:** SSH for the developer, cron, and read access to the GitLab project `root/digitalfirst`: a **deploy token** with the `read_repository` scope (GitLab → the project → Settings → Repository → Deploy tokens). The server can then fetch the code but never push.
- **Outbound HTTPS** from the server to `gitlab.fnri.dost.gov.ph` (the code), `challenges.cloudflare.com` (the "I'm human" check on sign-up), `oauth2.googleapis.com` and `www.googleapis.com` ("Continue with Google") and `api.pwnedpasswords.com` (the breached-password check). Without the Cloudflare and Google ones, sign-up and Google sign-in fail; testers' browsers reach Google and Cloudflare themselves.
- **A test SMTP account** for password-reset links and check-in notices.

B2. First deploy

Paths below are for Linux, with the code in `/var/www/elibrary`, a `deploy` user and the web server running as `www-data`.

1. Code, `.env` and the database

```
cd /var/www
git config --global credential.helper store          # keeps the deploy token in ~/.git-
credentials (mode 600)
git clone --branch staging https://gitlab.fnri.dost.gov.ph/root/digitalfirst.git elibrary
#   Username: the deploy token's username (gitlab+deploy-token-…)   Password: the token
cd elibrary
composer install --no-dev --optimize-autoloader
cp .env.example .env
php artisan key:generate
nano .env                                    # the staging column of D1
```

```
CREATE DATABASE elibrarydb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'elibrary'@'localhost' IDENTIFIED BY '<a strong password>';
GRANT ALL PRIVILEGES ON elibrarydb.* TO 'elibrary'@'localhost';
```

2. Data: either an empty library with the sample accounts,

```
php artisan migrate --seed --force
```

or a copy of the development database (to test with real records):

```
mysql -u elibrary -p elibrarydb < elibrarydb.sql      # made with: mysqldump --single-
transaction --routines elibrarydb
php artisan migrate --force                           # then D5, if the copy predates any of its
fixes
```

With the copy, bring the files too: `storage/app/` and `public/frontpages/` (covers). A development database can hold real patrons' details: keep staging inside the network (B3).

3. Files, search and caches

```
php artisan storage:link
php artisan fulltext:make-private          # full-text PDFs -> storage/app/private/pdf (safe to
re-run)
php artisan scout:sync-index-settings
php artisan scout:import "App\Models\ViewCatalog"
php artisan sitemap:generate
php artisan optimize                       # caches config, routes, views and events
php artisan permission:cache-reset
```

4. File permissions

```
sudo chown -R deploy:www-data /var/www/elibrary
sudo chmod -R g+rwX storage bootstrap/cache public/frontpages
sudo find storage bootstrap/cache public/frontpages -type d -exec chmod g+s {} +
sudo chmod g+w public public/robots.txt     # the sitemap (nightly, and Settings → Rebuild
sitemap) writes here
```

From now on run `php artisan` as the web server user (`sudo -u www-data php artisan …`), so the files it writes (logs, caches, the sitemap) stay writable for the site.

5. Web server, PHP and scheduler: the vhost or server block in D2 with the staging host name and certificate, the PHP settings in D3, and the cron line in D4.

B3. Keep staging private

- Only the FNRI network or VPN reaches it (firewall).
- Admin → **Settings** → **Search engines**: switch on **Ask search engines not to list this site**. Every admin page then shows a reminder.
- `APP_ENV=staging`, `APP_DEBUG=false`: error pages show no code or file paths.

B4. Accounts

1. Sign in as `superadmin@example.org` with `SEED_USER_PASSWORD`, then remove that key from `.env`.
2. Give the super admin a real email and a new password (profile menu → Profile).
3. Add an account for each tester (Admin → Accounts → Administrators → Add), with the right role.
4. Change the developer account's password; delete the sample patron when testers have their own.

B5. Check it

Run the smoke test in D6.

B6. Update staging

1. Release: on the developer's PC. The `gitlab` remote is added once: `git remote add gitlab https://gitlab.fnri.dost.gov.ph/root/digitalfirst.git`.

```
git fetch origin
git checkout staging
git pull gitlab staging                                # anything pushed to staging from
elsewhere
git merge --no-ff origin/main -m "Release to staging"
git push gitlab staging                                # the pre-push hook runs the checks first
git checkout -                                         # back to the branch you were on
```

GitHub's workflows don't run on GitLab: the pre-push hook (A7) is the check before a push to staging, so keep the development packages installed on the PC that pushes.

2. Deploy: on the staging server.

```
cd /var/www/elibrary
sudo -u www-data php artisan down --retry=60           # visitors see "back soon" (503) meanwhile
git pull                                               # the staging branch, from GitLab
composer install --no-dev --optimize-autoloader
sudo -u www-data php artisan migrate --force
sudo -u www-data php artisan optimize
sudo -u www-data php artisan permission:cache-reset
sudo systemctl reload php8.5-fpm                       # drops PHP's cached code
sudo -u www-data php artisan up
```

Then any one-time command the release names (D5), and the smoke test.

B7. Start staging over

`php artisan migrate:fresh --seed --force`, then the search import. **This deletes every table and its data:** staging only, never production.

Part C — Production

Production is Part B with the differences below. Deploy every release to staging first.

C1. How production differs from staging

	Staging	Production
`APP_ENV`	`staging`	`production`
Address	internal host name	the public domain: one canonical host (www **or** not), the other redirects to it
Who reaches it	the FNRI network or VPN	everyone
Search engines	asked to stay away (Settings switch on)	welcome (switch **off**), sitemap, Google Search Console
Mail	test SMTP account	the library's SMTP account and sender address
Accounts	sample and test accounts	named staff accounts only; no sample accounts
`ADMIN_EMAIL`	any	the library's address (the built-in default belongs to another institute)
"I'm human" check	Cloudflare's test keys	the agency's own Turnstile widget for the domain (D10)
"Continue with Google"	the local-and-staging OAuth client; only its test users can sign in	the production OAuth client; the app published, so anyone can
Backups	optional	daily, and a restore tested (D7)

C2. Before go-live

- [ ] Domain and DNS; certificate; HTTP → HTTPS redirect; one canonical host.
- [ ] HTTPS works on that host **and every subdomain of it** before the first HTTPS visit: the app sends HSTS with `includeSubDomains`, and browsers then refuse plain HTTP there for a year.
- [ ] Behind a load balancer or proxy that ends TLS: list it in `app/Http/Middleware/TrustProxies.php` (`$proxies`), or Laravel sees plain HTTP and sends no HSTS.
- [ ] `.env`: the production column of D1.
- [ ] `php.ini`: the production settings in D3.
- [ ] Web server: D2; scheduler: D4.
- [ ] Data: an empty library (`migrate --seed`, then replace the sample accounts as in B4), or the staging or development database (dump, `migrate --force`, D5).
- [ ] Backups running and one restore tested (D7).
- [ ] AI mode decided (D9): off, the local AI only, or a Gemini option (approved under the agency's data policy, with a billing-enabled key).
- [ ] The production Turnstile widget and Google OAuth client, and the Google app published (D10); no test keys in `.env`.
- [ ] The smoke test passes (D6).

C3. First deploy and go-live

1. Part B2, steps 1 to 5, with production values.
2. Admin → Settings → Search engines: **Ask search engines not to list this site** stays **off**.
3. `php artisan sitemap:generate` once `APP_URL` is the real `https://` domain (the nightly run keeps it current).
4. Verify the site in Google Search Console and submit the sitemap (D8).

C4. Updating production

1. Deploy the same commit to staging and test it there.
2. Back up the database and files (D7).
3. Part B6 on the production server.
4. The smoke test.

> **Never on production:** `migrate:fresh`, `migrate:reset` or `db:wipe` (they delete every table; the migration history includes `2025_09_15_114348_drop_all_tables`), and **`db:seed`**, which empties the accounts, roles, permissions and settings tables and loads the sample data in their place.

C5. Rolling back

When an update goes wrong, restore rather than reverse the migrations:

```
sudo -u www-data php artisan down
git checkout <the previous commit>
composer install --no-dev --optimize-autoloader
mysql -u elibrary -p elibrarydb < backup-before-update.sql
sudo -u www-data php artisan optimize
sudo systemctl reload php8.5-fpm
sudo -u www-data php artisan up
```

Part D — Reference

D1. Environment variables

`.env.example` lists every key with a comment. The ones that differ by environment:

Key	Local	Staging	Production
`APP_NAME`	`eLibrary`	`eLibrary`	`eLibrary`
`APP_ENV`	`local`	`staging`	`production`
`APP_DEBUG`	`true`	`false`	**`false`**
`APP_URL`	`http://localhost:8000`	`https://<staging host>`	`https://<domain>`
`APP_KEY`	`php artisan key:generate`	its own	its own: never share it between environments
`ADMIN_EMAIL`	optional	a test address	the library's address
`DB_DATABASE`	`elibrarydb`	`elibrarydb`	`elibrarydb`
`DB_USERNAME`, `DB_PASSWORD`	your account	`elibrary` + password	`elibrary` + password
`SEED_USER_PASSWORD`	while seeding only	while seeding only	while seeding only
`SESSION_DRIVER`	`database`	`database`	`database`
`SESSION_LIFETIME`	`180`	`180`	`180` (above the 2-hour idle limit)
`SESSION_SECURE_COOKIE`	`false` only on `http://elibrary.test`	`true`	`true`
`SESSION_DOMAIN`	empty	empty	empty
`MAIL_MAILER`	`log`	`smtp`	`smtp`
The other `MAIL_*` keys	—	the test SMTP account	the library's SMTP account
`MAIL_FROM_ADDRESS`	any	a test sender	the library's sender
`SCOUT_DRIVER`	`meilisearch`	`meilisearch`	`meilisearch`
`MEILISEARCH_HOST`	`http://127.0.0.1:7700`	the same	the same
`MEILISEARCH_KEY`, `MEILI_MASTER_KEY`	a random value	its own	its own (16+ characters)
`MEILI_ENV`	`development`	`production`	`production`
`HERMES_GATEWAY_URL`, `_TOKEN`	if AI mode is used	if AI mode is used	if AI mode is used
`PASSWORD_BREACH_CHECK`	`true` (`false` offline)	`true` (`false` without internet)	`true`

Key	Local	Staging	Production
`TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	Cloudflare's test keys	Cloudflare's test keys	the agency's widget
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	the local-and-staging client	the local-and-staging client	the production client
`GOOGLE_REDIRECT_URI`	empty	empty	empty (the site's address + `/auth/google/callback`)
`HSTS_MAX_AGE`	`31536000`	`31536000`	`31536000` (`0` to back out of HTTPS)
`CSP_ENFORCE`	`true`	`true`	`true` (`false` = emergency switch)
`LOG_LEVEL`	`debug`	`debug`	`warning`

After changing `.env` on a server, run `php artisan optimize` again: with cached config, Laravel never reads `.env` at run time.

D2. Web server

The web root is the app's **`public/` folder** and nothing above it. Only `public/index.php` may run as PHP; folder listings stay off. Uploads are up to 100 MB (full-text PDFs), and Settings → Rebuild search index can take minutes.

Apache 2.4 (`sudo a2enmod rewrite headers ssl proxy_fcgi setenvif`; `public/.htaccess` does the rest: the front controller, the only-`index.php` rule, `nosniff` and HSTS on static files):

```
<VirtualHost *:80>
    ServerName elibrary.example.gov.ph
    Redirect permanent / https://elibrary.example.gov.ph/
</VirtualHost>

<VirtualHost *:443>
    ServerName elibrary.example.gov.ph
    DocumentRoot /var/www/elibrary/public

    SSLEngine on
    SSLCertificateFile    /etc/ssl/certs/elibrary.crt
    SSLCertificateKeyFile /etc/ssl/private/elibrary.key
    SSLProtocol -all +TLSv1.2 +TLSv1.3

    <Directory /var/www/elibrary/public>
        AllowOverride All
        Options -Indexes
        Require all granted
    </Directory>

    <FilesMatch "\.php$">
        SetHandler "proxy:unix:/run/php/php8.5-fpm.sock|fcgi://localhost"
    </FilesMatch>

    # 110 MB: full-text PDFs up to 100 MB. 600 s: rebuilding the search index.
    LimitRequestBody 115343360
    ProxyTimeout 600
</VirtualHost>
```

nginx:

```
server {
    listen 80;
    server_name elibrary.example.gov.ph;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;               # nginx 1.25.1+: "listen 443 ssl;" and "http2 on;"
    server_name elibrary.example.gov.ph;
    root /var/www/elibrary/public;
    index index.php;

    ssl_certificate     /etc/ssl/certs/elibrary.crt;
    ssl_certificate_key /etc/ssl/private/elibrary.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    client_max_body_size 110m;          # full-text PDFs up to 100 MB
    autoindex off;
    add_header X-Content-Type-Options nosniff always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /index.php {             # the only PHP that runs
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        fastcgi_param DOCUMENT_ROOT   $realpath_root;
        fastcgi_pass unix:/run/php/php8.5-fpm.sock;
        fastcgi_read_timeout 600;       # rebuilding the search index
    }

    location ~ \.php$ { return 404; }
    location ~ /\.(?!well-known) { deny all; }
}
```

Full-text PDFs live in `storage/app/private/pdf`, outside `public/`: only the app's Read and Download (signed in, published records) and the staff preview serve them. Never link or copy that folder into `public/`.

D3. PHP settings

In `php.ini` (Linux: both `/etc/php/8.5/fpm/php.ini` and `/etc/php/8.5/cli/php.ini`; Laragon: `C:\laragon\bin\php\php-8.5.12-Win32-vs17-x64\php.ini`), then reload PHP-FPM or Laragon's Apache. Start a server from PHP's own `php.ini-production`.

Setting	Every environment	Production and staging	Why
`upload_max_filesize`	`100M`		full-text PDFs up to 100 MB (MARC files 40 MB, covers 10 MB)
`post_max_size`	`110M`		the upload plus the form
`memory_limit`	`1024M`		batch import and search rebuild (development PCs use `2048M`)

Setting	Every environment	Production and staging	Why
`max_execution_time`	`600`		rebuilding the search index
`session.upload_progress.enabled`	`0`		otherwise anyone can make PHP write text they choose into a session file
`display_errors`, `display_startup_errors`		`Off`	no file paths or SQL on screen (set `APP_DEBUG=false` too)
`expose_php`		`Off`	no `X-Powered-By: PHP/8.5.12` header
`session.use_strict_mode`		`1`	PHP refuses session IDs it didn't create
`allow_url_include`		`Off` (the default)	code is never included from a URL
`opcache.enable`		`1`	speed; reload PHP-FPM after each deploy

Keep **one** PHP installed (8.5.12 or newer). Check: `php -i | grep -E "upload_progress.enabled|expose_php|display_errors|use_strict_mode|upload_max_filesize"` (Windows: `findstr` in place of `grep -E`).

D4. Scheduler

It runs `visitors:forget` at 03:15, `sitemap:generate` at 03:30 and `search:reconcile` at 03:45 (`app/Console/Kernel.php`). Without it guests' IP addresses are kept past the 90 days the privacy notice promises, the sitemap goes stale and the nightly search check never runs.

- **Linux:** in the web server user's crontab (`sudo crontab -u www-data -e`):

  ```
  * * * * * cd /var/www/elibrary && php artisan schedule:run >> /dev/null 2>&1
  ```

- **Windows:** once, `powershell -ExecutionPolicy Bypass -File scripts\register-scheduler-task.ps1` (add `-Php "<path to php.exe>"` if PHP is not on PATH). It registers the `eLibrary-scheduler` task.
- **Check:** `php artisan schedule:list` shows the three jobs.

D5. Moving an existing database forward

For a database copied from another environment. Run `migrate` every time; the rest once, if the copy predates the change. Each `--apply` command shows what it would change when run without `--apply`, and is safe to run again.

What	Command
New tables, columns and permissions (and their grants to roles)	`php artisan migrate --force`, then `php artisan permission:cache-reset`; review Admin → Accounts → Roles
Full-text PDFs still under `public/pdf`	`php artisan fulltext:make-private`

What	Command
Material types saved from the leader byte (LDR/06)	`php artisan catalog:sync-material-types --apply`
Titles showing only MARC 245 `$a`	`php artisan catalog:rebuild-titles --apply`
Authors missing for corporate (110) or added-entry (700/710) names	`php artisan catalog:rebuild-authors --apply`
Search page without subjects, years, filters or sorts	`php artisan catalog:rebuild-search-fields --apply` (also syncs the index settings and re-imports)
Records without a cover image	`php artisan covers:placeholders` (generated covers; `--clear` removes them)
MARC reference tables from before October 2026	below

Replacing **only** the three MARC reference tables with the corrected seed data (never `db:seed` on a database with real accounts):

```
php artisan tinker --execute='DB::statement("SET FOREIGN_KEY_CHECKS=0"); foreach
(["marctags_loc", "marctag_indicators_loc", "marctag_subfields_loc"] as $t) { DB::table($t)-
>truncate(); foreach
(array_chunk(json_decode(file_get_contents(database_path("seeders/data/$t.json")), true), 500)
as $rows) DB::table($t)->insert($rows); echo $t, ": ", DB::table($t)->count(), PHP_EOL; }
DB::statement("SET FOREIGN_KEY_CHECKS=1");'
```

Then rebuild the search index: `php artisan scout:sync-index-settings` and `php artisan scout:import "App\Models\ViewCatalog"`.

D6. Smoke test

After every deploy (replace `<site>` with the address):

```
php artisan about --only=environment       # Environment as expected; Debug Mode OFF on servers
php artisan migrate:status --pending       # "No pending migrations"
php artisan schedule:list                  # sitemap:generate 03:30, search:reconcile 03:45
curl -sI https://<site>/home | grep -iE "^HTTP|strict-transport|content-security"
curl -s "https://<site>/search/suggest?q=rice"   # {"query":"rice","total":…} from Meilisearch
curl -sI https://<site>/sitemap.xml | head -1    # 200
```

In the browser:

- Sign in; **Admin** → **Settings** → **Tools** → **Check search** says "Search is up to date".
- The OPAC search (`/results?q=…`) finds records; a record page opens, with its cover.
- Upload a cover on staging, and see it in the record and the OPAC.
- A wrong address shows the eLibrary's own "not found" page, with no code or file paths.
- `/register` shows the "I'm human" check, which passes (staging: it says "For testing only"; production: it must not).
- "Continue with Google" on `/login` signs in a test user (staging) or any Google account (production) as a patron.

D7. Backups and restore

What	Where	How often
The database	`elibrarydb`	daily: `mysqldump --single-transaction --routines elibrarydb | gzip > elibrarydb-$(date +%F).sql.gz`
Uploaded files	`storage/app/` (full texts in `private/pdf`, thumbnails, news covers) and `public/frontpages/` (covers)	daily
Settings	`.env`	after each change, stored apart from the backups, as it holds the secrets

The search index needs no backup: `scout:import` rebuilds it from the database. Test a restore before go-live: load the latest dump into a scratch database and open the app against it.

D8. Google Search Console

Production only, after go-live. It shows how the ~7,600 record pages are indexed and which searches bring people. The sitemap can only be submitted once the site is verified.

1. Sign in to https://search.google.com/search-console with an **institutional** Google account (the account that adds the site owns it); add colleagues as users afterwards.
2. **Add property** → **URL prefix**: `https://<domain>`.
3. Choose **HTML tag**; copy only the code inside `content="…"` (it is public, not a secret).
4. Admin → **Settings** → **Search engines** → **Google Search Console verification code**, then **Save changes** (a whole pasted tag keeps only its code).
5. Back in Search Console: **Verify**, then **Sitemaps** → submit `https://<domain>/sitemap.xml`.

Google can verify only once the site is publicly reachable; leave the code in place, as Google checks again from time to time. A DNS TXT record is the alternative that doesn't depend on the app.

D9. The AI half (optional)

AI mode answers OPAC questions from the collection. It needs the `hermes-with-rag` stack: a model server, the AI gateway on port 8090 and a sync worker, on a machine with a GPU. Its own guides (`DEPLOYMENT_RUNBOOK.md`, `CATALOG_SYNC_RUNBOOK.md`, `STARTING_HERMES_GUIDE.md`) cover installing it.

- **The eLibrary's side:** `HERMES_GATEWAY_URL` and `HERMES_GATEWAY_TOKEN` in `.env` (the token must match the gateway's), then Admin → Settings → AI answers. The status box there says whether the gateway answers.
- **Shared:** the gateway and the sync worker read the eLibrary's MySQL database and the full-text PDFs (`storage/app/private/pdf`).
- **Who writes the answers** (Admin → Settings → AI answers): the local AI only (nothing leaves the server), or a Google Gemini option (the question and its passages go to Google: this needs approval under the agency's data policy and a billing-enabled key, kept in the AI server's `.env`, never here).
- **Runs by itself** once `register-projecty-tasks.ps1` (in `hermes-with-rag`, run elevated) has registered its tasks: the model, the gateway and the sync worker at logon, and the nightly AI-index reconcile at 02:30.

- **Check:** `curl http://127.0.0.1:8090/healthz` on the gateway's machine; Admin → AI Sync Monitor shows the pending changes falling to 0.

Without the AI half, leave AI answers off: the rest of the eLibrary works without it.

D10. Google sign-in and the "I'm human" check

Patrons may sign up and sign in with "Continue with Google" (no password, no email verification: Google has confirmed the address), and the sign-up form has an "I'm human" check (Cloudflare Turnstile) in place of email verification. Staff accounts keep their passwords. How it works: `SECURITY_NOTES.md` section 2.

The Google Cloud project (once). With the Google account that will own it (preferably an agency account, so it outlives any one person's), at https://console.cloud.google.com:

1. Create a project, e.g. "DOST-FNRI eLibrary".
2. **Google Auth Platform** → **Branding** (the "OAuth consent screen"): app name "DOST-FNRI eLibrary", a support email, the developer's contact email. **Audience:** External.
3. **Data access:** the scopes `openid`, `…/auth/userinfo.email`, `…/auth/userinfo.profile` and nothing else. These need no review by Google.
4. **Audience** → **Test users:** each tester's Google address (up to 100). While the app is in **Testing**, only they can sign in; anyone else sees "access blocked".
5. **Clients** → **Create client** → **Web application**, "eLibrary local and staging", with the authorized redirect URIs `http://localhost:8000/auth/google/callback` and `https://<staging host>/auth/google/callback` (no JavaScript origins needed). Copy the client ID and the secret into `.env` as `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`; the console shows the secret only now, so download the JSON as well and keep it out of git.
6. **For production:** a second client, "eLibrary production", with `https://<domain>/auth/google/callback` only; its secret goes only on the production server. Before go-live, **Audience** → **Publish app**, so any Google account can sign in. Adding a logo to the consent screen needs Google's brand check (a few days); without one it is not needed.

Cloudflare Turnstile.

- **Local and staging:** Cloudflare's test keys, no account needed (the `.env.example` values). Site key `1x00000000000000000000AA` with secret `1x0000000000000000000000000000000AA` pass every check; site key `2x00000000000000000000AB` with secret `2x0000000000000000000000000000000AA` fail every check. The widget says "For testing only".
- **Production:** a free Cloudflare account → **Turnstile** → **Add widget**: name "eLibrary", the hostname of the domain, widget mode **Managed**. Its site key and secret go in `.env`.

After changing these keys on a server: `php artisan optimize`.

Testing it.

- Automated (no Google account or internet; part of the pre-push checks): `php artisan test --filter='GoogleSignInTest|HumanCheckTest'`. Google's and Cloudflare's answers are faked.
- By hand, as a test user, locally on `http://localhost:8000` or on staging:

Try	Expect
Continue with Google, with an address that has no account	signed in as a new patron with a library code; Profile says "Set a Password"
The same, with the address of an existing patron	signed in to that account; its password still works
The same, with a staff account's address	back on the sign-in page: "Staff accounts sign in with their email and password, not with Google."
A banned patron's address	the banned popup, as with a password
Cancel on Google's page	back on the sign-in page: "Google sign-in was cancelled."
Profile → Set a Password, 15 minutes or more after signing in	"Confirm with Google" first, then the password form
Sign up with the test keys that fail (`2x…`)	"The “I’m human” check didn’t pass." and no account

D11. Troubleshooting

What you see	Cause and fix
Signing in says "Page expired" (419), or comes back to the sign-in page	The browser refused the session cookie: plain HTTP with `SESSION_SECURE_COOKIE=true` (use HTTPS, or `localhost` ; on `http://elibrary.test` set it to `false`), or `SESSION_DOMAIN` set to another host. Then `php artisan optimize:clear` (local) or `php artisan optimize` (servers)
"Page expired" on a form after a long pause	The session ended (staff after 30 minutes idle, patrons after 2 hours): sign in again
Search says "temporarily unavailable", or 0 results	Meilisearch is down (`curl http://127.0.0.1:7700/health`), the key is wrong, or its settings were never synced: `php artisan scout:sync-index-settings` , then `scout:import` . Admin → Settings → Tools → Check search compares the index with the database
Covers or thumbnails missing	`php artisan storage:link` not run, or the web server can't write `public/frontpages` and `storage` (B2 step 4)
Uploads of large PDFs fail	`upload_max_filesize` , `post_max_size` (D3) or the web server's body limit (D2) too low
500 error after an update	Read `storage/logs/laravel.log` ; usually file permissions or a stale cache: `php artisan optimize:clear` , then `php artisan optimize`
Menus missing for a role after an update	`php artisan permission:cache-reset` ; then check the role in Admin → Accounts → Roles (a module's switch must be on for the permissions inside it)
The sitemap lists the wrong address	It was built before `APP_URL` was right: Admin → Settings → Tools → Rebuild sitemap
`git pull` on staging asks for a password, or says "Authentication failed"	The deploy token expired or was revoked: make a new one in GitLab (B1), then update `~/.git-credentials`
No mail arrives	`MAIL_*` in `.env` ; with `MAIL_MAILER=log` mail is only written to the log
A `.env` change has no effect	Config is cached: `php artisan optimize`
Google says "Error 400: redirect_uri_mismatch"	The OAuth client doesn't list this exact return address: the site's address (as the browser has it) + `/auth/google/callback` , `http` only for `localhost` (D10)

What you see	Cause and fix
Google says "Access blocked: … has not completed the Google verification process"	The app is in Testing and this Google account isn't a test user: add it, or publish the app (production)
"Google sign-in didn't finish. Please try again."	The session was lost on the way: the site was opened on one host and Google returned to another (`localhost` and `127.0.0.1`), or the server can't reach `oauth2.googleapis.com` ; the log has the reason
"The “I’m human” check couldn’t be completed just now"	The server can't reach `challenges.cloudflare.com` (B1); the log says why
The log says the "I'm human" check "is set up wrongly"	`TURNSTILE_SECRET_KEY` is wrong, or isn't the secret of `TURNSTILE_SITE_KEY`
The "I'm human" check says "For testing only" in production	Cloudflare's test keys are still in `.env` : put in the agency's widget keys

D12. Known gaps

- **Some new records may not be citable in AI mode.** The sync worker stores their passages under `catalog-{id}` , while `AiSearchService::resolveSources()` resolves sources by accession number. Found in code review, not yet reproduced.